

SAC

Soft Actor-Critic (SAC) Algorithm

Table of Contents

1. [Background and Motivation](#)
 2. [The Maximum Entropy Objective](#)
 3. [Soft Value Functions](#)
 4. [Soft Policy Iteration](#)
 5. [Function Approximation](#)
 6. [The Tanh Correction Term](#)
 7. [Automatic Entropy Tuning](#)
 8. [The Partition Function \$Z\(s\)\$ in Practice](#)
 9. [Actor-Critic Architecture Analysis](#)
 10. [Twin Q-Networks and Overestimation Bias](#)
 11. [Off-Policy vs On-Policy Analysis](#)
 12. [Implementation Details and Engineering Tricks](#)
 13. [Practical Projects](#)
 14. [Training Diagnostics](#)
 15. [Study Quiz](#)
 16. [Appendix A: Mathematical Foundations](#)
 17. [Appendix B: Symbol Table](#)
 18. [Appendix C: Core Formula Summary](#)
-

Chapter 1: Background and Motivation

1.1 Limitations of Standard Reinforcement Learning

Standard reinforcement learning algorithms aim to maximize the expected cumulative reward:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

where $\tau = (s_0, a_0, s_1, a_1, \dots)$ is a trajectory sampled from the policy π .

Limitations:

1. **Poor Exploration:** Standard RL agents tend to converge to local optima quickly, especially in sparse reward environments. Without explicit encouragement to explore, the agent may miss better strategies.
2. **Sensitivity to Initial Conditions:** The learned policy can be heavily dependent on the initial state distribution and early experiences.
3. **Lack of Robustness:** A policy that is optimal for one task distribution may completely fail when the environment changes slightly.

1.2 Maximum Entropy Reinforcement Learning Framework

Soft Actor-Critic (SAC), derived by Tuomas Haarnoja et al. in 2018, is an **off-policy actor-critic algorithm** based on the **maximum entropy reinforcement learning framework**.

Unlike standard RL, which only maximizes expected cumulative rewards, SAC encourages the agent to act as randomly as possible while still succeeding at the task:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right]$$

This leads to much better exploration and robustness.

1.3 Why Entropy Regularization?

The entropy $\mathcal{H}(\pi)$ of a policy measures the "randomness" or "uncertainty" in the action selection:

- **High entropy** $\implies$ The policy is more exploratory, spreading probability mass across many actions.
- **Low entropy** $\implies$ The policy is more deterministic, focusing on specific actions.

Benefits:

1. **Explicit Exploration Incentive:** The agent is explicitly rewarded for maintaining uncertainty, preventing premature convergence to suboptimal deterministic policies.
2. **Multi-Modal Discovery:** In tasks with multiple equally good strategies, maximum entropy RL can learn to use all of them rather than committing to one.
3. **Improved Stability:** The entropy term smooths the optimization landscape, making gradient-based updates more stable.

Chapter 2: The Maximum Entropy Objective

2.1 Standard RL Objective

In standard RL, the goal is to maximize the expected sum of rewards:

$$J(\pi) = \mathbb{E}_{(s_t,a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

where ρ_π is the state-action trajectory distribution under policy π .

2.2 Entropy-Augmented Objective

In maximum entropy RL, we add an entropy regularization term to the objective function. The **generalized objective** $J(\pi)$ is defined as:

$$J(\pi) = \mathbb{E}_{(s_t,a_t) \sim \rho_\pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$$

Symbol Definitions:

Symbol	Meaning
ρ_π	State-action trajectory distribution under policy π
α	Temperature parameter controlling the trade-off between reward and entropy
$\mathcal{H}(\pi(\cdot s_t))$	Entropy of the policy distribution at state s_t

2.3 The Temperature Parameter α

The temperature parameter α controls the **trade-off** between:

- **Maximizing reward** (exploitation): Focusing on actions known to yield high returns.
- **Maximizing entropy** (exploration): Acting as randomly as possible.

Effect of α values:

α Value	Behavior
$\alpha = 0$	Reduces to standard RL; no entropy bonus
$\alpha \rightarrow \infty$	Policy becomes purely random (uniform distribution)
α small but positive	Balances exploitation and exploration

2.4 Shannon Entropy Definition

The Shannon entropy of the policy at state s_t is calculated as:

$$\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[-\log \pi(a_t|s_t)]$$

Intuition: The entropy is the expected negative log-probability of the action taken. If the policy is deterministic (always selects one action with probability 1), then $-\log(1) = 0$ and entropy is zero. If the policy is uniform (all actions equally likely), entropy is maximized.

Chapter 3: Soft Value Functions

3.1 Soft Q-Function: Definition and Intuition

The soft Q-value represents the expected return **plus** the expected future entropy of following policy π after taking action a_t in state s_t :

$$Q(s_t, a_t) = r(s_t, a_t) + \mathbb{E}_{\tau \sim \pi} \left[\sum_{k=1}^{T-t} \gamma^k (r(s_{t+k}, a_{t+k}) + \alpha \mathcal{H}(\pi(\cdot|s_{t+k}))) \right]$$

Key insight: Unlike standard Q-values which only count future rewards, soft Q-values also account for future entropy bonuses. This means the agent is rewarded for maintaining exploration options even in the future.

3.2 Soft V-Function: Definition and Intuition

The soft V-value represents the expected total value (including entropy) of starting in state s_t :

$$V(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)]$$

Symbol Definitions:

Symbol	Meaning
$Q(s_t, a_t)$	Soft Q-function: value of taking action a_t in state s_t
$V(s_t)$	Soft V-function: value of being in state s_t
$\alpha \log \pi(a_t s_t)$	Entropy bonus scaled by temperature

3.3 Relationship Between Soft Q and Soft V

By substituting the definition of $V(s_{t+1})$ into the Q-function, we can derive:

$$V(s_t) = \mathbb{E}_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log \pi(a_t | s_t)]$$

This shows that the soft V-function is the **expectation** of the soft Q-function under the current policy, minus the entropy term.

3.4 The Soft Bellman Equation

By substituting $V(s_{t+1})$ back into the Q-function, we derive the **Soft Bellman Equation**:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p}[V(s_{t+1})]$$

Comparison with Standard Bellman Equation:

Standard RL	Soft RL
$Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s', a'}[Q(s', a')]$	$Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s'}[V(s')]$
Q-function depends on future Q-values	Q-function depends on V-function (expectation over actions)

The key difference is that the soft V-function already includes the entropy term, so we don't need to sample an action in the backup — we only need the expected next state value.

Chapter 4: Soft Policy Iteration

4.1 Overview: Two-Step Alternating Process

SAC guarantees convergence to the optimal maximum entropy policy by **alternating** between two steps:

1. **Soft Policy Evaluation:** Compute the soft Q-value for an arbitrary policy π .
2. **Soft Policy Improvement:** Update the policy toward the optimal distribution.

This is analogous to the standard Policy Iteration algorithm, but adapted for the maximum entropy framework.

4.2 Soft Policy Evaluation: The Contraction Operator

We want to compute the soft Q-value for an arbitrary policy π . We define a **modified Bellman backup operator** $\mathcal{T}^\pi$:

$$\mathcal{T}^\pi Q(s, a) \triangleq r(s, a) + \gamma \mathbb{E}_{s' \sim p}[V(s')]$$

where $V(s') = \mathbb{E}_{a' \sim \pi}[Q(s', a') - \alpha \log \pi(a' | s')]$.

Contraction Mapping Theorem:

By applying the contraction mapping theorem (using the supremum norm), it can be mathematically proven that repeatedly applying $\mathcal{T}^\pi$ causes Q to **converge monotonically** to the true soft Q-value Q^π for that policy.

4.3 Soft Policy Improvement: Constrained Optimization

In the improvement step, we want to find a new policy π_{new} that **maximizes** the expected soft value for a given $Q^{\pi_{\text{old}}}$:

$$\pi_{\text{new}} = \arg \max_{\pi' \in \Pi} \mathbb{E}_{a \sim \pi'} [Q^{\pi_{\text{old}}}(s, a) - \alpha \log \pi'(a|s)]$$

Constrained Optimization Problem:

We need to solve this optimization over a **tractable space of policies** Π (like Gaussians), not over all possible distributions.

4.4 The Partition Function $Z(s)$ and Normalization

The Boltzmann (Gibbs) distribution assigns probabilities exponentially proportional to Q-values:

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

To make this a valid probability distribution, we need to normalize:

$$\pi^*(a|s) = \frac{\exp\left(\frac{1}{\alpha} Q(s, a)\right)}{Z(s)}$$

where $Z(s)$ is the **partition function**:

$$Z(s) = \int_{\mathcal{A}} \exp\left(\frac{1}{\alpha} Q(s, a')\right) da'$$

Why We Need $Z(s)$:

An exponential function $\exp(\frac{1}{\alpha} Q(s, a))$ outputs values from 0 to ∞ . These are "unnormalized scores." For a valid probability density, the total integral must equal 1:

$$\int_{\mathcal{A}} \pi(a|s) da = 1$$

4.5 KL Divergence Projection (Information Projection)

To solve the constrained optimization, we **project** the exponential of the soft Q-function onto Π using the **Kullback-Leibler (KL) divergence**:

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot|s) \parallel \frac{Z(s) \exp \left(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, \cdot) \right)}{Z(s)} \right)$$

From the Original SAC Article:

"To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence."

Decoding this sentence:

1. **"The constraint $\pi \in \Pi$ ":** Our neural network can only output simple shapes (e.g., Gaussian distributions). The perfect theoretical policy π^* might be a complex, multi-modal shape that our network cannot represent.
2. **"Project":** Find the closest distribution within Π to the theoretical optimal π^* .
3. **"Information projection":** Use KL divergence in a specific direction ($D_{\text{KL}}(\pi_{\phi} \parallel \pi^*)$) rather than other distance metrics.

4.6 Mathematical Proof of Monotonic Improvement

Expanding the KL Divergence:

$$D_{\text{KL}}(\pi' \parallel \pi^*) = \mathbb{E}_{a \sim \pi'} \left[\log \pi'(a|s) - \frac{1}{\alpha} Q^{\pi_{\text{old}}}(s, a) + \log Z(s) \right]$$

Since $\log Z(s)$ depends only on the state (not on π'), it acts as a **constant** during optimization.

Multiplying the inside by α :

$$D_{\text{KL}} = \alpha \cdot \mathbb{E}_{a \sim \pi'} \left[\frac{1}{\alpha} \log \pi'(a|s) - \frac{1}{\alpha^2} Q^{\pi_{\text{old}}}(s, a) + \frac{1}{\alpha} \log Z(s) \right]$$

Minimizing this KL divergence is mathematically identical to maximizing:

$$\mathbb{E}_{a \sim \pi'} [Q^{\pi_{\text{old}}}(s, a) - \alpha \log \pi'(a|s)]$$

This proves that updating the policy via KL divergence minimization strictly improves its soft value function.

Chapter 5: Function Approximation

5.1 Neural Network Architecture

In continuous, high-dimensional spaces, we cannot compute these equations exactly. Instead, we use neural networks:

Network	Symbol	Function
Critic	$Q_{\theta}(s, a)$	Approximates the soft Q-value
Actor	$\pi_{\phi}(a s)$	Outputs the policy distribution

5.2 Critic Network Loss (Q-Network)

The parameters θ are trained by minimizing the **mean squared Bellman error**:

$$J_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\theta}(s, a) - y)^2 \right]$$

where the **target value function** $\hat{V}(s')$ is:

$$\hat{V}(s') = \mathbb{E}_{a' \sim \pi_{\phi}(\cdot|s')} [Q_{\bar{\theta}}(s', a') - \alpha \log \pi_{\phi}(a'|s')]$$

Target Value Calculation:

$$y = r + \gamma \hat{V}(s')$$

Symbol Definitions:

Symbol	Meaning
$\mathcal{D}$	Replay buffer containing past transitions
$Q_{\bar{\theta}}$	Target Q-network (slowly updated)
γ	Discount factor

5.3 Actor Network Loss (π -Network)

The actor parameters ϕ are trained directly to minimize the KL divergence objective:

$$J_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\phi}} [\alpha \log \pi_{\phi}(a|s) - Q_{\theta}(s, a)]$$

Key Points:

- States s are sampled from the **replay buffer** (off-policy data).
- Actions a are sampled from the **current policy** π_{ϕ} .
- The gradient flows through Q_{θ} back to update the actor.

5.4 The Reparameterization Trick

Problem: The stochastic sampling $a \sim \pi_\phi(\cdot|s)$ creates a **high-variance** gradient estimator because:

$$\nabla_\phi \mathbb{E}_{a \sim \pi_\phi}[f(a)] = \mathbb{E}_{a \sim \pi_\phi}[f(a) \nabla_\phi \log \pi_\phi(a)]$$

This "log-likelihood" trick has high variance because it depends on the gradient of the policy itself.

Solution: Apply the **reparameterization trick**. We define the action as a **deterministic function** of the state and an independent noise vector ϵ :

$$a = f_\phi(\epsilon; s) = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I)$$

Why this works:

1. The noise ϵ is independent of ϕ .
2. Gradients can now flow directly through μ_ϕ and σ_ϕ because the stochasticity is isolated in ϵ .

Reparameterized Actor Loss:

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, \epsilon \sim \mathcal{N}} [\alpha \log \pi_\phi(f_\phi(\epsilon; s)|s) - Q_\theta(s, f_\phi(\epsilon; s))]$$

Chapter 6: The Tanh Correction Term

6.1 Why We Need Tanh Squashing

Neural networks naturally output unbounded values ($-\infty$ to $+\infty$). However, physical environments have **hard constraints**:

- Robot joints move between specific angle limits
- Motor torques have maximum and minimum values
- Action values must be scaled to environment limits

To enforce this, SAC passes the raw Gaussian policy output through a **tanh activation function** to bound actions to $[-1, 1]^D$.

6.2 Change-of-Variables Formula

Problem: Because the action distribution is squashed by tanh, we cannot simply calculate the log-likelihood using the standard Gaussian formula. We must use the **probability change-of-variables formula**.

Setup:

1. $u \in \mathbb{R}^D$: Raw unbounded action sampled from Gaussian distribution $\mu(u|s)$
2. $a = \tanh(u)$: Squashed, final action bounded within $(-1, 1)^D$

The probability density of the squashed action $\pi(a|s)$ is related to the base density $\mu(u|s)$ by:

$$\pi(a|s) = \mu(u|s) \cdot \left| \det \left(\frac{du}{da} \right) \right|^{-1}$$

where $\frac{du}{da}$ is the **Jacobian matrix** of the transformation.

6.3 Jacobian Determinant for Diagonal Gaussian

Because $\tanh$ is applied **element-wise** to each action dimension independently, the Jacobian is a **diagonal matrix**:

$$\frac{du_i}{da_i} = \frac{1}{\text{sech}^2(u_i)} = 1 - \tanh^2(u_i)$$

The determinant of a diagonal matrix is simply the product of its diagonal elements:

$$\det \left(\frac{du}{da} \right) = \prod_{i=1}^D (1 - \tanh^2(u_i))$$

6.4 Deriving the Final Log-Likelihood

Plugging this back into the change-of-variables equation:

$$\pi(a|s) = \mu(u|s) \cdot \prod_{i=1}^D \frac{1}{1 - \tanh^2(u_i)}$$

Taking the natural logarithm:

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^D \log (1 - \tanh^2(u_i))$$

This is the exact correction term used in SAC.

6.5 Stable Numerical Implementation

Problem: If you try to calculate $\log(1 - \tanh^2(u))$ directly in PyTorch or TensorFlow, your code will crash with NaN errors whenever u is large.

Reason: When $u = 10$, $\tanh(10) \approx 1$, making $1 - \tanh^2(u) \approx 0$, and $\log(0) \rightarrow -\infty$.

Solution: Use trigonometric identities to rewrite the correction term in a **stable form**:

$$1 - \tanh^2(u) = \frac{4}{(e^u + e^{-u})^2}$$

Taking the log:

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u)$$

where $\text{softplus}(x) = \log(1 + e^x)$.

Practical Implementation:

```
log_prob = normal.log_prob(x_t) - torch.log(1 - action.pow(2) + 1e-6)
```

The `1e-6` is a small epsilon to prevent numerical instability when the action is exactly at the boundary.

6.6 Confusion Point: Why the Correction is Necessary

Common Confusion: Some practitioners forget to apply the tanh correction, which leads to:

1. **Incorrect Policy Gradient:** The gradients don't properly account for the bounded action space.
2. **Policy Collapse:** The agent may output actions outside the valid range when the environment clips them.

Why it works mathematically: The change-of-variables formula ensures that probability mass is conserved when we transform the distribution. The log determinant term accounts for how tanh "stretches" or "compresses" probability density in different regions.

Chapter 7: Automatic Entropy Tuning

7.1 Constrained Formulation of Maximum Entropy RL

Instead of fixing α as a constant, we can reframe maximum entropy RL as a **constrained optimization problem**:

$$\max_{\pi_{0:T}} \mathbb{E} \left[\sum_{t=0}^T r(s_t, a_t) \right] \quad \text{s.t.} \quad \mathcal{H}(\pi_t(\cdot|s_t)) \geq \bar{\mathcal{H}} \quad \forall t$$

Goal: Maximize expected rewards subject to the constraint that the policy maintains a minimum average entropy level $\bar{\mathcal{H}}$.

7.2 Lagrange Multiplier Method

Using the method of **Lagrange Multipliers**, we convert this constrained objective into an unconstrained **dual problem**:

$$\mathcal{L}(\pi_{0:T}, \alpha_{0:T}) = \mathbb{E} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha_t (\mathcal{H}(\pi_t(\cdot|s_t)) - \bar{\mathcal{H}}) \right]$$

where α_t is our dual variable (Lagrange multiplier) at time t .

7.3 The α Update Loss Function

By expanding the entropy term $\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \sim \pi}[-\log \pi(a_t|s_t)]$, the objective function for the temperature parameter at time t simplifies to:

$$J(\alpha) = \alpha \cdot \mathbb{E}_{a_t \sim \pi_\phi} \left[-\log \pi_\phi(a_t|s_t) - \bar{\mathcal{H}} \right]$$

In code:

```
alpha_loss = -(self.log_alpha * (log_prob +
self.target_entropy).detach()).mean()
```

7.4 Dynamics of α Adaptation (Corrected Explanation)

To see how α changes, we take the derivative of $J(\alpha)$ with respect to α :

$$\frac{\partial J}{\partial \alpha} = \mathbb{E}_{a \sim \pi}[-\log \pi_\phi(a|s)] - \bar{\mathcal{H}} = \mathcal{H}(\pi) - \bar{\mathcal{H}}$$

In network training, we update parameters using **gradient descent** (moving opposite to the gradient):

$$\alpha \leftarrow \alpha - \eta \frac{\partial J}{\partial \alpha}$$

where η is the learning rate.

Scenario 1: Actual Entropy is Too High

- Condition: $-\log \pi_\phi(a_t|s_t) > \bar{\mathcal{H}}$ on average, meaning $\mathcal{H}(\pi) > \bar{\mathcal{H}}$
- Gradient Sign: $\frac{\partial J}{\partial \alpha} > 0$ (positive)
- The Update: Because we subtract a positive gradient, α **decreases**
- The Result: A lower α reduces the entropy penalty weight. The agent dials back exploration to focus more on pure rewards (exploitation).

Scenario 2: Actual Entropy is Too Low

- Condition: $-\log \pi_\phi(a_t|s_t) < \bar{\mathcal{H}}$ on average, meaning $\mathcal{H}(\pi) < \bar{\mathcal{H}}$
- Gradient Sign: $\frac{\partial J}{\partial \alpha} < 0$ (negative)
- The Update: Subtracting a negative gradient means α **increases**
- The Result: A higher α heavily penalizes predictable actions, forcing the policy to widen its distribution and explore more.

Summary:

$-\log \pi(a|s) > \bar{\mathcal{H}} \implies$ **Entropy is too high** $\implies \alpha$ **decreases**.

$-\log \pi(a|s) < \bar{\mathcal{H}} \implies$ **Entropy is too low** $\implies \alpha$ **increases**.

7.5 Choosing Target Entropy $\bar{\mathcal{H}}$

In practice, setting a universal entropy value is challenging. Haarnoja et al. discovered a simple heuristic:

$$\bar{\mathcal{H}} = -\dim(\mathcal{A})$$

where $\dim(\mathcal{A})$ is the number of dimensions in the action space.

Intuition: For a diagonal Gaussian policy with D independent action dimensions, the entropy of a uniform distribution over $[-1, 1]^D$ scales with the action space dimension. This heuristic works well across many continuous control tasks.

Chapter 8: The Partition Function $Z(s)$ in Practice

8.1 Physics Origin: Boltzmann Distribution

The concept of a **partition function** comes from statistical mechanics and thermodynamics.

In maximum entropy RL, we want our ideal policy $\pi^*(a|s)$ to assign higher probabilities to actions that yield higher Q-values:

$$\pi^*(a|s) \propto \exp\left(\frac{1}{\alpha} Q(s, a)\right)$$

This is known as the **Boltzmann (or Gibbs) distribution**. In physics, particles occupy lower energy states with higher probability. If we map "Energy" to "Negative Q-value" ($E = -Q$), the mathematics are identical.

8.2 Why $Z(s)$ Cannot Be Computed Directly

Problem: Calculating the integral across continuous action spaces is **analytically intractable**:

$$Z(s) = \int_{\mathcal{A}} \exp\left(\frac{1}{\alpha} Q(s, a')\right) da'$$

Reason: We cannot solve this integral because:

1. The action space $\mathcal{A}$ is continuous and potentially infinite.
2. The Q-function $Q(s, a')$ is a neural network — we don't have a closed-form expression.

8.3 The Gradient Vanishing Trick

Key Insight: We don't need to compute $Z(s)$ because it acts as a **constant** relative to the policy parameters ϕ .

Expanding the KL divergence objective:

$$J_{\pi}(\phi) = \mathbb{E}_{a \sim \pi_{\phi}} \left[\log \pi_{\phi}(a|s) - \frac{1}{\alpha} Q(s, a) + \log Z(s) \right]$$

Since $Z(s)$ depends only on s and Q , it does **not** contain ϕ .

When we take the gradient:

$$\nabla_{\phi} J_{\pi}(\phi) = \nabla_{\phi} \mathbb{E}_{a \sim \pi_{\phi}} \left[\log \pi_{\phi}(a|s) - \frac{1}{\alpha} Q(s, a) \right] + \nabla_{\phi} \log Z(s)$$

Because $Z(s)$ does not depend on ϕ :

$$\boxed{\nabla_{\phi} \log Z(s) = 0}$$

The partition function naturally vanishes during differentiation.

8.4 Connection to Information Projection

This is precisely why the authors chose the **information projection** (I-projection, $D_{\text{KL}}(\pi_{\phi} \parallel \pi^*)$) rather than other projections:

- With I-projection: $Z(s)$ becomes an additive constant $\implies$ vanishes during gradient update.
- With alternative projections (e.g., reverse KL): $Z(s)$ would remain trapped inside an expectation over actions sampled from π^* , requiring the impossible integral to be computed.

Chapter 9: Actor-Critic Architecture Analysis

9.1 Why Both Q-Network and V-Network? (Original Design)

In the original 2018 SAC paper, the authors included a **separate V-network** to reduce computational variance and stabilize updates.

Equation 8 from the original paper:

$$\hat{Q}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\psi}}(s_{t+1})]$$

Benefit: By having a dedicated V-network, calculating the target for the Q-network doesn't require sampling an action a_{t+1} from the policy at every training step.

9.2 Why One Network Is Enough (SAC-v2 Evolution)

Your intuition is correct: One network is enough!

In a subsequent revision (often called **SAC-v2**), the authors realized that maintaining three separate networks (Actor, Q-critic, V-critic) was **mathematically redundant**.

In modern SAC, we only use a Q-network. The target is calculated by sampling the next action directly:

$$J_Q(\theta) = \mathbb{E} \left[\frac{1}{2} (Q_\theta(s, a) - (r + \gamma(Q_{\bar{\theta}}(s', a') - \alpha \log \pi_\phi(a'|s'))))^2 \right]$$

9.3 The Circular Dependency Problem

Question: The update of V-network relies on the estimate of Q, and that of Q also relies on V. Isn't this severely unstable?

Answer: This circular dependency is called **bootstrapping** — updating an estimate based on another estimate. Left unchecked, this causes values to explode or diverge.

SAC's Circular Dependency:

1. **Q updates based on V** (Equation 5): V estimates the value of the next state.
2. **V updates based on Q** (Equation 8): Q is trained to match the V -based targets.

9.4 Target Network and Polyak Averaging (Stability Mechanism)

Safety Valve: SAC uses a **Target Network** $V_{\bar{\psi}}$ (or $Q_{\bar{\theta}}$ in modern versions) that is **frozen** or changing very slowly.

Polyak Averaging (Soft Updates):

$$\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$$

where τ is a very small number (typically $\tau = 0.005$).

Stabilization Effect:

Network	Update Speed	Role
Q_θ (online)	Fast	Provides stable targets to the Q-network
$V_{\bar{\psi}}$ (target)	Slow (Polyak)	Tracks the true Q-value, preventing explosion

This design turns what would be a chaotic feedback loop into a **highly damped system**.

Chapter 10: Twin Q-Networks and Overestimation Bias

10.1 The Maximization Bias Problem

Your intuition is correct: The Q-network update is fundamentally biased because of a phenomenon called **Maximization Bias**.

Setup: A neural network $Q_\theta(s, a)$ naturally has estimation error:

$$Q_\theta(s, a) = Q^*(s, a) + \epsilon$$

where Q^* is the true value and ϵ is random noise with $\mathbb{E}[\epsilon] = 0$ (unbiased).

The Problem: In SAC, the actor is trained to maximize Q_θ . When selecting the action to maximize:

$$\mathbb{E} \left[\max_a Q_\theta(s, a) \right] \geq \max_a \mathbb{E}[Q_\theta(s, a)] = \max_a Q^*(s, a)$$

Due to **Jensen's Inequality**, the maximum of noisy estimates is **biased upward**. The actor preferentially selects actions where the noise ϵ happens to be positive (overestimated).

Analogy: Imagine asking 5 guessing friends how much a house is worth. Their individual errors are unbiased. If you take the **average**, you get an accurate estimate. But if you only listen to the friend who gives the **highest number**, your final estimate will always be heavily overestimated.

10.2 Mathematical Analysis of Error Propagation

Step $t + 1$: The actor selects an action that overestimates the value of state s_{t+1} due to a localized fluke in the neural network weights.

Step t : The Q-network updates its value for s_t by bootstrapping off that **inflated** s_{t+1} target.

The Loop: Now, the value for s_t is officially overestimated. When the agent is at state s_{t-1} , it bootstraps off this new inflated value.

Overestimation error cascades backward and multiplies, causing Q-values to grow infinitely large.

10.3 Why Accuracy Doesn't Cascade Cleanly

You asked an excellent question: Isn't accuracy transmitted from the latter timestep to the former?

Answer: In theory, yes. But in Reinforcement Learning with **bootstrapping**, we update our current estimate based on our **own future estimates**. This turns the transmission of data into a dangerous feedback loop.

In a perfect world, accuracy would transmit cleanly backward. In practice, overestimation error loops endlessly.

10.4 The Minimum Q-Value Trick (Clipped Double-Q)

To break this loop, SAC adopts the **Clipped Double-Q Learning** mechanism (originally from TD3). We initialize two **independent** Q-networks:

$$Q_{\theta_1}(s, a) \quad \text{and} \quad Q_{\theta_2}(s, a)$$

When calculating the target, we take the **minimum**:

$$y = r + \gamma \left(\min_i Q_{\theta_i}(s', a') - \alpha \log \pi_{\phi}(a'|s') \right)$$

Why the Minimum Works:

Since Q_{θ_1} and Q_{θ_2} are independent, their estimation errors ϵ_1 and ϵ_2 are also independent.

If Q_{θ_1} experiences a random variance spike and overestimates an action, it is **highly unlikely** that Q_{θ_2} will experience that exact same error on the exact same action at the exact same time.

By taking the minimum, the algorithm **filters out positive noise peaks**.

10.5 Why Underestimation Is Safer Than Overestimation

Mathematical Property:

$$\mathbb{E}[\min(X, Y)] \leq \min(\mathbb{E}[X], \mathbb{E}[Y])$$

Consequence: Taking the minimum shifts the bias from positive (overestimation) to neutral or slightly **negative** (underestimation).

Why underestimation is safer:

Bias Type	Agent Behavior	Outcome
Overestimation	Agent becomes overly confident in bad actions	Gets stuck repeating suboptimal actions, believing they are optimal
Underestimation	Agent continues to explore alternative actions	May miss some short-term reward but finds better long-term strategies

An underestimation bias is vastly safer for a reinforcement learning agent. If the agent underestimates a state slightly, it simply continues to explore. But if it overestimates, it will get stuck forever.

Chapter 11: Off-Policy vs On-Policy Analysis

11.1 Why SAC Can Be Off-Policy

SAC can use a replay buffer containing data from ancient, dead policies. Let's examine why.

The Critic Doesn't Care Who Took the Action:

SAC's critic evaluates using the **Soft Bellman Equation**:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})]$$

Key observation: The Bellman equation states a fundamental truth about the universe: if you are in state s_t and execute action a_t , the resulting reward and next state s_{t+1} depend **entirely on the physics of the environment**.

It doesn't matter if that action was chosen by the current smart policy, a random policy from training hour one, or a human playing the game. The transition (s_t, a_t, r_t, s_{t+1}) is a **ground-truth physical fact**.

The Actor Evaluates Old States with New Actions:

The actor's loss function:

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}} [\mathbb{E}_{a \sim \pi_\phi} [\alpha \log \pi_\phi(a|s) - Q(s, a)]]$$

Key observation:

1. States s are sampled from the **historical replay buffer** $\mathcal{D}$.
2. Actions a are sampled from the **current policy** π_ϕ .

When updating the actor, SAC looks at an old state from the buffer, ignores the action the agent actually took back then, and asks its current actor: "What action would **you** take here right now?"

11.2 Why PPO Is Inherently On-Policy

PPO belongs to the **Policy Gradient** family. It doesn't use the Bellman equation to optimize its policy; it directly estimates the gradient of expected rewards.

The Importance Sampling Trap:

To optimize using data collected by an older policy π_{old} , policy gradient methods use **Importance Sampling**:

$$r_t(\theta) = \frac{\pi_{\text{old}}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)}$$

The PPO objective:

$$J_{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right]$$

Problem: For this correction to have low variance, π_{θ} must be **incredibly close** to π_{old} .

If you feed PPO data from a replay buffer collected 100 iterations ago, π_{old} will be drastically different from π_{θ} :

- For many actions, $\pi_{\text{old}}(a|s) \approx 0$ while $\pi_{\theta}(a|s)$ is high
- The ratio $r_t(\theta)$ explodes toward infinity
- PPO's clipping mechanism violently forces these massive ratios back down
- Gradients become flat (zero) or carry catastrophic bias

11.3 The Importance Sampling Trap in PPO

Consequence: PPO requires its data to be **pristine** and collected by a policy that is **practically identical** to the one being updated.

This is why PPO must throw away data after each update — it's fundamentally on-policy.

11.4 Can PPO Be Modified to Off-Policy?

Answer: Yes, theoretically — but it effectively **stops being PPO**.

To build an "Off-Policy PPO," you would have to implement massive modifications:

Modification A: Trust Region/Variance Tracking

Replace clipping with advanced off-policy multi-step correction algorithms:

- **V-trace** (used in IMPALA)
- **Retrace**(λ)

These mathematically bound and truncate importance sampling weights across sequential trajectories.

Modification B: State Distribution Correction

PPO assumes states are sampled from the trajectory distribution of the current policy. Off-policy data suffers from **state distribution shift**. You'd need to calculate:

$$w(s) = \frac{\rho_{\pi_{\text{behavior}}}(s)}{\rho_{\pi_{\theta}}(s)}$$

Estimating this density ratio in high-dimensional continuous spaces is **notoriously difficult and computationally expensive**.

Verdict: If you make these modifications, you end up creating algorithms like **ACER** or **IMPALA** — which completely discard the simplicity and stability that made PPO popular.

11.5 Key Architectural Differences Summary

Property	SAC	PPO
Policy Type	Off-policy	On-policy
Data Reuse	Replay buffer (any past data)	Must discard after update
Value Estimation	Bellman equation with critics	Generalized Advantage Estimation (GAE)
Gradient Source	Critic network backpropagation	Importance sampling ratio
Stability	Target networks + Twin-Q	Clipping mechanism
Sample Efficiency	High (reuses data)	Low (wastes data)

Chapter 12: Implementation Details and Engineering Tricks

12.1 Why Predict log_std Instead of std

A standard deviation σ must be **strictly positive** ($\sigma > 0$). If your neural network outputs std directly via a standard linear layer, it could easily output a negative number during early training.

Passing a negative number into a Gaussian distribution will crash your code.

Solution: Force the network to predict the **logarithm** of the standard deviation ($\log \sigma$). A neural network's output layer can safely span any real number from $-\infty$ to $+\infty$.

When we compute $\sigma = \exp(\log \sigma)$, the exponential function **mathematically guarantees** a strictly positive value:

$$\exp(x) > 0 \quad \forall x \in \mathbb{R}$$

12.2 Clamping log_std: [-20, 2] Rationale

Even though $\exp(x)$ is always positive, computers have **finite precision**. Unbounded outputs lead to catastrophic failure modes:

Exploding Variance (log_std_max = 2):

- If the network outputs $\log \sigma = 20$, then $\sigma = \exp(20) \approx 4.8 \times 10^8$
- Your policy becomes so scattered that actions become pure random noise
- All learned behavior is erased

Numerical Underflow (log_std_min = -20):

- If the network outputs $\log \sigma = -50$, then $\sigma = \exp(-50) \approx 2.06 \times 10^{-9}$
- When calculating $\log \pi(a|s)$, we divide by σ or take $\log(\sigma)$
- Dividing by zero or $\log(0)$ generates **NaN errors**

Clamping Solution:

```
log_std = torch.clamp(log_std, self.log_std_min, self.log_std_max)
```

Clamping $\log \sigma$ between $[-20, 2]$ keeps σ safely bounded between approximately 2.06×10^{-9} and 7.39.

12.3 The .sum(dim=-1, keepdim=True) Operation

When an environment has multiple action dimensions (e.g., controlling a robot with 6 independent joint motors), your actor network outputs a mean μ_i and standard deviation σ_i for **each** individual dimension.

SAC assumes diagonal Gaussian policy: Action dimensions are statistically independent.

Joint Probability: The joint probability of multiple independent events is the **product** of their individual probabilities:

$$P(a_1, a_2, \dots, a_D) = \prod_{i=1}^D P(a_i)$$

In log-space: Products become sums:

$$\log P(a_1, a_2, \dots, a_D) = \sum_{i=1}^D \log P(a_i)$$

In PyTorch:

```
log_prob = log_prob.sum(dim=-1, keepdim=True)
```

- `dim=-1` sums across the action dimension (last axis)
- `keepdim=True` preserves the 2D shape (`batch_size`, 1) for subsequent matrix operations

12.4 Exploration vs Evaluation Mode

The `evaluate` flag switches between two modes:

```
def select_action(self, state, evaluate=False):
    state = torch.FloatTensor(state).unsqueeze(0)
    if evaluate:
        with torch.no_grad():
            mu, _ = self.actor(state)
            action = torch.tanh(mu)
            return action.numpy()[0]
    else:
        with torch.no_grad():
            action, _ = self.actor.sample(state)
            return action.numpy()[0]
```

When `evaluate=False` (Training Mode):

- The code calls `self.actor.sample(state)`
- Uses the reparameterization trick to sample an action randomly based on both μ and σ
- **Purpose:** Adds exploratory noise; the agent needs this randomness to discover high-reward states

When `evaluate=True` (Testing/Evaluation Mode):

- The code bypasses σ and sampling entirely
- Extracts only the **mean** μ and passes it through tanh
- **Purpose:** Makes the policy completely **deterministic** for testing. We want the agent to display its best behavior, not random exploration.

12.5 Complete PyTorch Implementation Walkthrough

```
import random
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.distributions import Normal
import gymnasium as gym

# --- REPLAY BUFFER ---
class ReplayBuffer:
    def __init__(self, capacity, state_dim, action_dim):
        self.capacity = capacity
        self.ptr = 0
        self.size = 0

        self.states = np.zeros((capacity, state_dim), dtype=np.float32)
        self.actions = np.zeros((capacity, action_dim), dtype=np.float32)
        self.rewards = np.zeros((capacity, 1), dtype=np.float32)
        self.next_states = np.zeros((capacity, state_dim), dtype=np.float32)
        self.dones = np.zeros((capacity, 1), dtype=np.float32)

    def push(self, state, action, reward, next_state, done):
        self.states[self.ptr] = state
        self.actions[self.ptr] = action
        self.rewards[self.ptr] = reward
        self.next_states[self.ptr] = next_state
        self.dones[self.ptr] = done

        self.ptr = (self.ptr + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, batch_size):
        ind = np.random.randint(0, self.size, size=batch_size)
        return (
            torch.FloatTensor(self.states[ind]),
            torch.FloatTensor(self.actions[ind]),
            torch.FloatTensor(self.rewards[ind]),
            torch.FloatTensor(self.next_states[ind]),
        )
```

```

        torch.FloatTensor(self.dones[ind])
    )

# --- NETWORKS ---
class CriticNetwork(nn.Module):
    """Twin Q-networks inside a single module for clean optimization."""
    def __init__(self, state_dim, action_dim, hidden_dim=256):
        super(CriticNetwork, self).__init__()
        # Q1
        self.q1 = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )
        # Q2
        self.q2 = nn.Sequential(
            nn.Linear(state_dim + action_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, state, action):
        sa = torch.cat([state, action], dim=-1)
        return self.q1(sa), self.q2(sa)

class ActorNetwork(nn.Module):
    def __init__(self, state_dim, action_dim, hidden_dim=256, log_std_min=-20,
log_std_max=2):
        super(ActorNetwork, self).__init__()
        self.log_std_min = log_std_min
        self.log_std_max = log_std_max

        self.net = nn.Sequential(
            nn.Linear(state_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU()
        )
        self.mu = nn.Linear(hidden_dim, action_dim)
        self.log_std = nn.Linear(hidden_dim, action_dim)

    def forward(self, state):
        x = self.net(state)
        mu = self.mu(x)
        log_std = self.log_std(x)
        log_std = torch.clamp(log_std, self.log_std_min, self.log_std_max)
        return mu, log_std

```

```

def sample(self, state):
    mu, log_std = self.forward(state)
    std = log_std.exp()

    # Reparameterization trick
    normal = Normal(mu, std)
    x_t = normal.rsample()
    action = torch.tanh(x_t)

    # Tanh Log-Likelihood Correction
    log_prob = normal.log_prob(x_t) - torch.log(1 - action.pow(2) + 1e-6)
    log_prob = log_prob.sum(dim=-1, keepdim=True)

    return action, log_prob

# --- SAC AGENT ---
class SACAgent:
    def __init__(self, state_dim, action_dim, lr=3e-4, gamma=0.99, tau=0.005):
        self.gamma = gamma
        self.tau = tau
        self.action_dim = action_dim

        # Critics
        self.critic = CriticNetwork(state_dim, action_dim)
        self.critic_target = CriticNetwork(state_dim, action_dim)
        self.critic_target.load_state_dict(self.critic.state_dict())
        self.critic_opt = optim.Adam(self.critic.parameters(), lr=lr)

        # Actor
        self.actor = ActorNetwork(state_dim, action_dim)
        self.actor_opt = optim.Adam(self.actor.parameters(), lr=lr)

        # Automatic Entropy Tuning
        self.target_entropy = -action_dim
        self.log_alpha = torch.zeros(1, requires_grad=True)
        self.alpha_opt = optim.Adam([self.log_alpha], lr=lr)

    @property
    def alpha(self):
        return self.log_alpha.exp()

    def select_action(self, state, evaluate=False):
        state = torch.FloatTensor(state).unsqueeze(0)
        if evaluate:

```

```

        with torch.no_grad():
            mu, _ = self.actor(state)
            action = torch.tanh(mu)
            return action.numpy()[0]
    else:
        with torch.no_grad():
            action, _ = self.actor.sample(state)
            return action.numpy()[0]

def update(self, memory, batch_size):
    state, action, reward, next_state, done = memory.sample(batch_size)

    # 1. Update Critic
    with torch.no_grad():
        next_action, next_log_prob = self.actor.sample(next_state)
        target_q1, target_q2 = self.critic_target(next_state, next_action)
        target_q = torch.min(target_q1, target_q2) - self.alpha *
next_log_prob
        expected_q = reward + (1 - done) * self.gamma * target_q

    curr_q1, curr_q2 = self.critic(state, action)
    critic_loss = nn.MSELoss()(curr_q1, expected_q) + nn.MSELoss()
(curr_q2, expected_q)

    self.critic_opt.zero_grad()
    critic_loss.backward()
    self.critic_opt.step()

    # 2. Update Actor
    new_action, log_prob = self.actor.sample(state)
    q1, q2 = self.critic(state, new_action)
    min_q = torch.min(q1, q2)

    actor_loss = (self.alpha * log_prob - min_q).mean()

    self.actor_opt.zero_grad()
    actor_loss.backward()
    self.actor_opt.step()

    # 3. Update Alpha
    alpha_loss = -(self.log_alpha * (log_prob +
self.target_entropy).detach())).mean()

    self.alpha_opt.zero_grad()
    alpha_loss.backward()
    self.alpha_opt.step()

```

```

        # 4. Soft Update Targets
        for param, target_param in zip(self.critic.parameters(),
self.critic_target.parameters()):
            target_param.data.copy_(self.tau * param.data + (1 - self.tau) *
target_param.data)

# --- TRAINING LOOP ---
if __name__ == "__main__":
    env = gym.make("Pendulum-v1")
    state_dim = env.observation_space.shape[0]
    action_dim = env.action_space.shape[0]

    agent = SACAgent(state_dim, action_dim)
    buffer = ReplayBuffer(capacity=100000, state_dim=state_dim,
action_dim=action_dim)

    batch_size = 256
    episodes = 50

    for ep in range(episodes):
        state, _ = env.reset()
        ep_reward = 0
        done = False

        while not done:
            action = agent.select_action(state)
            # Scale action from [-1, 1] to environment limits
            scaled_action = action * env.action_space.high[0]

            next_state, reward, terminated, truncated, _ =
env.step(scaled_action)
            done = terminated or truncated

            buffer.push(state, action, reward, next_state, float(terminated))
            state = next_state
            ep_reward += reward

            if buffer.size > batch_size:
                agent.update(buffer, batch_size)

        print(f"Episode: {ep+1} | Reward: {ep_reward:.2f} | Alpha:
{agent.alpha.item():.4f}")
    env.close()

```

Chapter 13: Practical Projects

13.1 Quadcopter / Drone Control

The Task: Teach a drone to stabilize, track moving waypoints, or recover smoothly from sudden wind gusts.

Environment Toolkit: `gym-pybullet-drones` (built on PyBullet) or AirSim.

Why SAC fits: Drones require precise, continuous motor RPM adjustments. SAC's entropy exploration naturally discovers smoother control trajectories, minimizing structural stress compared to aggressive algorithms.

13.2 Autonomous Driving & Lane Keeping

The Task: Teach a simulated vehicle to navigate complex traffic, execute highway overtakes, and maintain lane discipline.

Environment Toolkit: CARLA Simulator (high-fidelity) or `highway-env` (fast, lightweight 2D simulator).

Why SAC fits: Driving relies on smooth throttle, braking, and steering inputs. The maximum entropy framework prevents the vehicle from getting trapped in local minima.

13.3 Bipedal Locomotion (Robotic Walking)

The Task: Coordinate multiple joints on a two-legged or four-legged robot to walk forward over rough terrain without falling.

Environment Toolkit: MuJoCo (`Ant-v4` , `Walker2d-v4` , or `Humanoid-v4`) via Gymnasium.

Why SAC fits: Locomotion features highly complex physics interactions. SAC is one of the few algorithms robust enough to successfully map coordinated continuous joint movements for top-heavy bipeds.

13.4 Financial Portfolio Optimization

The Task: Maximize returns while minimizing volatility by adjusting a continuous weighting vector across a pool of assets.

Environment Toolkit: FinRL framework or custom Gymnasium asset tracking code.

Why SAC fits: The action space is a softmax probability vector over continuous asset weights. SAC's entropy exploration acts as an **implicit risk-management tool**, preventing the model

from over-leveraging into a single high-performing asset.

Chapter 14: Training Diagnostics

14.1 Understanding Reward Vibration

The "huge vibration" in rewards during training is actually **not a failure to converge**. Your SAC agent has likely already learned the optimal policy!

The vibration is a combined result of how the environment is designed and how SAC samples actions during training.

14.2 The Role of Randomized Initial States

Your code is running on `Pendulum-v1`. In this environment, every single time an episode resets, the pendulum is spawned at a **completely random angle** ($\theta \in [-\pi, \pi]$) with random velocity.

The reward penalizes the angle away from upright:

$$r = -(\theta^2 + 0.1 \cdot \dot{\theta}^2 + 0.001 \cdot u^2)$$

The Single-Digit Episodes (e.g., -1.59): The pendulum happened to spawn almost perfectly upright. The agent didn't have to work to swing it up; it just held it still.

The High-Negative Episodes (e.g., -115.40): The pendulum spawned hanging straight down. Even a flawless policy must spend 20-40 timesteps swinging back and forth to build momentum before standing upright.

14.3 Training Mode vs Evaluation Mode

During training, the agent chooses actions by **sampling from its Gaussian distribution** (`evaluate=False`).

Even though α has decayed to a small value (making the policy narrow), it is still **stochastic**. A single poorly sampled action can cause the pendulum to twitch, lose balance, and drop.

14.4 How to Verify True Convergence

To see if your agent has actually converged, test it **deterministically** without exploratory noise:

```
# --- EVALUATION TEST ---  
eval_reward = 0
```

```
eval_state, _ = env.reset()
eval_done = False

while not eval_done:
    action = agent.select_action(eval_state, evaluate=True) # Use mean only
    scaled_action = action * env.action_space.high[0]

    next_state, reward, terminated, truncated, _ = env.step(scaled_action)
    eval_done = terminated or truncated
    eval_state = next_state
    eval_reward += reward

print(f"=== DETERMINISTIC EVALUATION REWARD: {eval_reward:.2f} ===")
```

What you will find: The deterministic evaluation rewards will be **consistently high and smooth**. The variance in training is a feature of the environment's randomized resets and SAC's active exploration, not an unstable model!

Chapter 15: Study Quiz

Question 1: Core Optimization Objective

What is the core optimization objective of Soft Actor-Critic (SAC) that distinguishes it from standard model-free reinforcement learning algorithms?

- A) Minimizing the tracking error of a reference trajectory.
- B) Maximizing expected return while also maximizing policy entropy.
- C) Maximizing the deterministic reward of the environment.
- D) Minimizing the variance of the state-value function updates.

Correct Answer: B

Explanation: SAC augments the standard reward maximization objective with an entropy term. This framework (Maximum Entropy RL) encourages exploration and robustness by rewarding the policy for acting randomly or maintaining high uncertainty where possible.

Question 2: The Temperature Parameter (α)

In SAC, the temperature parameter α controls the trade-off between exploration and exploitation. What happens if α is set to 0?

- A) The policy becomes purely random, prioritizing maximum entropy.
- B) The critic updates stop because the soft Q-value function becomes undefined.
- C) The algorithm reduces to a standard, non-entropy-augmented reinforcement learning algorithm.
- D) The actor will completely stop updating because gradients vanish.

Correct Answer: C

Explanation: Setting $\alpha = 0$ removes the entropy regularization term completely. As a result, the objective function simplifies back to standard expected reward maximization.

Question 3: Twin Q-Functions

Why does SAC typically use two Q-networks (Twin Q-functions) to approximate the action-value function?

- A) To mitigate the overestimation bias of the Q-values by taking the minimum of the two targets.
- B) To separately model the rewards from the environment and the intrinsic entropy rewards.
- C) To allow simultaneous on-policy and off-policy evaluation.
- D) To compute the policy gradient using a contrastive loss between two independent critics.

Correct Answer: A

Explanation: Adopted from the TD3 (Twin Delayed DDPG) algorithm, taking the minimum of two independent Q-value targets bounds the overestimation bias that commonly plagues actor-critic methods.

Question 4: Continuous Action Spaces

SAC is designed for continuous action spaces. How does the actor network typically output actions to allow for stochastic exploration?

- A) It outputs a deterministic action vector combined with Ornstein-Uhlenbeck noise.

- B) It outputs discrete logits representing a softmax probability distribution over action bins.
- C) It outputs a single scalar value corresponding to the expected value of the best action.
- D) It outputs the mean and standard deviation of a Gaussian distribution, from which actions are sampled.

Correct Answer: D

Explanation: In standard continuous SAC, the policy is parameterized as a squashed Gaussian distribution. The neural network outputs the mean and variance for each action dimension, allowing the agent to sample actions stochastically.

Question 5: Gradient Variance & Backpropagation

Which technique is used in SAC to compute low-variance gradients for the policy parameters through the stochastic sampling process?

- A) The REINFORCE score function estimator.
- B) The reparameterization trick (pathwise derivative estimator).
- C) Generalized Advantage Estimation (GAE).
- D) Importance sampling correction from an on-policy buffer.

Correct Answer: B

Explanation: The reparameterization trick expresses the stochastic action as a deterministic function of the policy parameters and an independent noise variable (e.g., $a = f_\phi(\epsilon, s)$). This allows gradients to flow directly from the critic back to the actor via standard backpropagation.

Question 6: Soft Q-Value Target Equation

In the soft Q-value update step of SAC, what is the target value function y for a transition (s, a, r, s') ?

- A) $y = r + \gamma[\min_{i=1,2} Q_{\theta_i}^{\text{target}}(s', a') - \alpha \log \pi_\phi(a'|s')]$, where $a' \sim \pi_\phi(\cdot|s')$.
- B) $y = r + \gamma \max_{a'} [\min_{i=1,2} Q_{\theta_i}(s', a')]$.

C) $y = r + \alpha \log \pi_{\phi}(a|s) + \gamma \max_{i=1,2} Q_{\theta_i}(s', a')$.

D) $y = r + \gamma[\frac{1}{2}Q_{\theta_1}(s', a') + Q_{\theta_2}(s', a')]$.

Correct Answer: A

Explanation: The target value incorporates the immediate environment reward, the discounted minimum soft Q-value of the next state (from target networks), and subtracts the scaled log-probability ($\alpha \log \pi$) of the next action to account for the entropy bonus.

Question 7: Automatic Temperature Adjustment

How is the temperature parameter α handled in modern implementations of SAC?

- A) It is decayed according to a predefined exponential schedule, similar to ϵ -greedy exploration.
- B) It is fixed as a hyperparameter that must be manually tuned for every environment.
- C) It is automatically adjusted over time by optimizing a loss function constrained to a target entropy.
- D) It is learned directly as an extra output head of the Critic network.

Correct Answer: C

Explanation: Modern SAC automatically tunes α dynamically by formulating a constrained optimization problem. The algorithm attempts to maximize rewards while keeping the policy's entropy above a user-specified target threshold.

Question 8: Algorithm Type & Memory

Is Soft Actor-Critic an on-policy or off-policy algorithm, and what component enables this?

- A) On-policy, enabled by calculating the soft policy gradient directly from the current rollout trajectories.
- B) Off-policy, enabled by a replay buffer that stores past experiences.
- C) Off-policy, enabled by updating the policy using importance sampling weights from prior policies.

D) On-policy, enabled by maintaining synchronized target networks that match the current state distributions.

Correct Answer: B

Explanation: SAC is an off-policy algorithm. It utilizes a replay buffer to store and reuse historical data transitions, making it drastically more sample-efficient than on-policy alternatives like PPO.

Question 9: Benefits of Maximum Entropy

What is a primary benefit of maximizing entropy in SAC's policy optimization?

- A) It encourages the policy to explore more widely, discover multiple optimal sub-policies, and prevent premature convergence.
- B) It ensures that the critic network converges faster by smoothing the reward landscape.
- C) It completely eliminates the need for target networks during value estimation.
- D) It reduces the memory requirements of the replay buffer by filtering out redundant states.

Correct Answer: A

Explanation: Explicitly maximizing entropy ensures the agent keeps its options open and continues exploring alternative actions. This prevents the policy from collapsing prematurely into a single local optimum.

Question 10: Target Network Updates

In SAC, how are the parameters of the target Q-networks updated?

- A) By copying the weights of the current Q-networks every fixed number of steps (hard updates).
- B) Directly via gradient descent using the mean squared error of the policy advantage.
- C) They are not updated online; they are trained offline after collecting an entire epoch of data.
- D) Through Polyak averaging (soft updates) using a small smoothing coefficient τ .

Correct Answer: D

Explanation: Target networks in SAC track the main online Q-networks via soft updates: $\theta_{\text{target}} \leftarrow \tau\theta + (1 - \tau)\theta_{\text{target}}$. This incremental tracking ensures smooth target estimations and stabilizes the bootstrapping process.

Appendix A: Mathematical Foundations

A.1 Kullback-Leibler Divergence Properties

The KL divergence $D_{\text{KL}}(P\|Q)$ measures the "distance" from distribution Q to distribution P :

$$D_{\text{KL}}(P\|Q) = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right] = \mathbb{E}_{x \sim P} [\log P(x) - \log Q(x)]$$

Key Properties:

1. **Non-negativity:** $D_{\text{KL}}(P\|Q) \geq 0$
2. **Zero iff identical:** $D_{\text{KL}}(P\|Q) = 0 \iff P = Q$
3. **Asymmetry:** $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$ in general
4. **Additivity:** For independent distributions,
$$D_{\text{KL}}(P_1 \times P_2\|Q_1 \times Q_2) = D_{\text{KL}}(P_1\|Q_1) + D_{\text{KL}}(P_2\|Q_2)$$

A.2 Change-of-Variables Formula (General Form)

For a transformation $a = f(u)$ where f is invertible:

$$\pi_A(a) = \pi_U(f^{-1}(a)) \cdot \left| \det \left(\frac{df^{-1}}{da} \right) \right|$$

Equivalently, using the inverse:

$$\pi_A(a) = \pi_U(u) \cdot \left| \det \left(\frac{df}{du} \right) \right|^{-1}$$

A.3 Softplus Function Stable Implementation

The softplus function is defined as:

$$\text{softplus}(x) = \log(1 + e^x)$$

Stable numerical implementation:

```
def stable_softplus(x):  
    # For large x, this is numerically stable  
    return torch.where(x > 20, x, torch.log1p(torch.exp(x)))
```

Connection to tanh correction:

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u)$$

Appendix B: Symbol Table

Symbol	Meaning
π	Policy function
$\pi_\phi(a s)$	Policy parameterized by neural network with parameters ϕ
ρ_π	State-action trajectory distribution under policy π
α	Temperature parameter controlling entropy-reward trade-off
$\mathcal{H}(\pi(\cdot s))$	Shannon entropy of policy at state s
$\bar{\mathcal{H}}$	Target entropy threshold
$Q(s, a)$	Soft Q-function: value of taking action a in state s
$V(s)$	Soft V-function: value of being in state s
γ	Discount factor
$\mathcal{T}^\pi$	Soft Bellman backup operator
$Z(s)$	Partition function for normalizing the Boltzmann distribution
$Q_\theta(s, a)$	Q-function parameterized by neural network with parameters θ
$Q_{\bar{\theta}}$	Target Q-network (slowly updated)
ϵ	Random noise vector for reparameterization trick
$\mu_\phi(s)$	Mean of the Gaussian policy
$\sigma_\phi(s)$	Standard deviation of the Gaussian policy
τ	Polyak averaging coefficient for soft target updates
$\mathcal{D}$	Replay buffer containing past transitions
$D_{\text{KL}}(P Q)$	Kullback-Leibler divergence from Q to P
$\text{softplus}(x)$	$\log(1 + e^x)$

Appendix C: Core Formula Summary

C.1 Maximum Entropy Objective

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)) \right]$$

C.2 Soft Bellman Equation

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p}[V(s_{t+1})]$$

C.3 Soft V-Function

$$V(s_t) = \mathbb{E}_{a_t \sim \pi}[Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)]$$

C.4 Reparameterization Trick

$$a = f_\phi(\epsilon; s) = \tanh(\mu_\phi(s) + \sigma_\phi(s) \odot \epsilon), \quad \epsilon \sim \mathcal{N}(0, I)$$

C.5 Tanh Log-Likelihood Correction

$$\log \pi(a|s) = \log \mu(u|s) - \sum_{i=1}^D \log(1 - \tanh^2(u_i))$$

C.6 Stable Tanh Correction

$$\log(1 - \tanh^2(u)) = 2 \log 2 - u - \text{softplus}(-2u)$$

C.7 Actor Loss

$$J_\pi(\phi) = \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_\phi} [\alpha \log \pi_\phi(a|s) - Q_\theta(s, a)]$$

C.8 Critic Loss

$$J_Q(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(s, a) - (r + \gamma(Q_{\bar{\theta}}(s', a') - \alpha \log \pi_\phi(a'|s'))))^2 \right]$$

C.9 Alpha Loss (Automatic Tuning)

$$J(\alpha) = \alpha \cdot \mathbb{E}_{a_t \sim \pi_\phi} [-\log \pi_\phi(a_t|s_t) - \bar{\mathcal{H}}]$$

C.10 Polyak Averaging (Soft Updates)

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta}$$

C.11 Twin Q-Minimum Trick

$$y = r + \gamma \left(\min_i Q_{\bar{\theta}_i}(s', a') - \alpha \log \pi_\phi(a'|s') \right)$$

Author: 千喜 Qx

Date: 2026/06/09

Based on materials from Haarnoja et al. (2018) and subsequent SAC-v2 revisions